

Projekt

Technologie webowe

Krzysztof Mizgała, album 262839

2024-12-20

Spis treści

Wprowadzenie	3
Cel projektu	3
Czym jest scrapowanie danych?	3
Dlaczego BoardGameGeek.com?	3
Przegląd technologii	3
Struktura projektu	4
Scrapowanie danych	4
Przygotowanie środowiska	4
Uruchomienie scrapera	5
Proces scrapowania	5
Logowanie do serwisu	5
Pobieranie linków	5
Pobieranie danych o grach	6
Zabezpieczenia i optymalizacja	6
Zapisywanie danych	7
Zakończenie	7
Analiza danych	7
Struktura danych	7
Czyszczenie i przetwarzanie danych	8
Analiza danych	9
Wyniki i wnioski	10

Modelowanie predycyjne	10
Przygotowanie danych	11
Podsumowanie	11

Wprowadzenie

Cel projektu

Projekt polega na scrapowaniu danych o grach planszowych z serwisu [BoardGameGeek](#) oraz analizie tych danych. Dane są pobierane za pomocą skryptu w Pythonie, który wykorzystuje bibliotekę Selenium do nawigacji po stronie internetowej i pobierania informacji o grach. Zebrane dane są następnie analizowane przy użyciu różnych technik analizy danych i uczenia maszynowego.

Czym jest scrapowanie danych?

Scrapowanie danych, znane również jako web scraping, to proces automatycznego pobierania informacji ze stron internetowych. Wykorzystuje się do tego specjalne skrypty lub narzędzia, które nawigują po stronach, pobierają zawartość i przetwarzają ją w celu uzyskania potrzebnych danych. Scrapowanie danych jest szczególnie przydatne, gdy dane nie są dostępne w formie łatwo dostępnych plików do pobrania, ale są wyświetlane na stronach internetowych.

Dlaczego BoardGameGeek.com?

BoardGameGeek.com (BGG) jest jednym z najbardziej znanych i szanowanych serwisów internetowych poświęconych grom planszowym. Strona ta jest szeroko uznawana za jedno z głównych źródeł informacji o grach planszowych, zarówno wśród graczy, jak i wydawców oraz projektantów gier. Dzięki swojej popularności, BGG gromadzi ogromną ilość danych na temat gier planszowych, co czyni go idealnym źródłem do analizy.

Przegląd technologii

- Technologie i narzędzia użyte w projekcie (Python, Selenium, Pandas, Matplotlib).
- Krótka charakterystyka wybranych bibliotek i frameworków.
- **Biblioteki Python:** Selenium, pandas, tqdm, dotenv.

- **Przeglądarka:** Chrome (lub inna wspierana przez Selenium, z odpowiednim WebDriverem).
- **Plik .env:** Przechowuje dane logowania (USER i PASS) do serwisu BoardGameGeek.

Struktura projektu

- `scraper.py`: Skrypt do scrapowania danych z BoardGameGeek.
- `projekt.ipynb`: Notebook Jupyter zawierający kod do analizy danych.
- `games.csv`: Plik CSV zawierający zebrane dane o grach planszowych.
- `raport.pdf`: Raport końcowy w formacie PDF.

Scrapowanie danych

Przygotowanie środowiska

Aby rozpocząć proces scrapowania danych, należy spełnić następujące wymagania:

1. **Python 3.11:** Upewnij się, że masz zainstalowaną wersję Python 3.11.
2. **Google Chrome:** Zainstaluj przeglądarkę Google Chrome.
3. **ChromeDriver:** Pobierz [ChromeDriver](#) i umieść go w katalogu zawierającym plik `scraper.py` lub dodaj jego lokalizację do zmiennej środowiskowej `PATH`.
4. **Biblioteki Python:** Zainstaluj wymagane biblioteki, wykonując poniższe polecenie:

```
pip install -r requirements.txt
```

5. **Utwórz plik .env:** Utwórz plik `.env` w katalogu projektu i dodaj do niego swoje dane logowania do BoardGameGeek:

```
USER='twoja_nazwa_uzytkownika'  
PASS='twoje_haslo'
```

Uruchomienie scrapera

Aby uruchomić scraper i pobrać dane o grach planszowych, wykonaj poniższe polecenie w terminalu:

```
python scraper.py
```

Scraper posiada również następujące opcje wiersza poleceń (flagi):

- `-g` lub `--games`: liczba gier do pobrania (domyślnie 10000)
- `-c` lub `--checkpoint`: zapisuj dane co określoną liczbę gier (domyślnie 100)
- `-f` lub `--filename`: nazwa pliku do zapisu danych (domyślnie `games.csv`)

Na przykład, aby pobrać dane o 5000 grach, zapisywać dane co 50 gier w pliku `my_games.csv`:

```
python scraper.py -g 5000 -c 50 -f my_games.csv
```

Proces scrapowania

Logowanie do serwisu

- Skrypt `scraper.py` inicjalizuje obiekt klasy `Scraper`, który konfiguruje przeglądarkę Chrome w trybie headless (bez interfejsu graficznego).
- Po wejściu na stronę scraper automatycznie akceptuje pliki cookie.
- Scraper otwiera panel logowania BoardGameGeek. Logowanie jest konieczne, aby uzyskać pełen dostęp do danych.
- Logowanie odbywa się za pomocą danych użytkownika (nazwa użytkownika i hasło) pobranych z pliku `.env`.
- Selenium symuluje wypełnienie pól logowania oraz kliknięcie przycisku „Zaloguj”.

Pobieranie linków

Metoda `get_links` pobiera linki do stron z grami planszowymi. Na każdej podstronie rankingu wyświetlane jest 100 gier. W celu uzyskania wystarczającego zbioru danych do analizy, pobieramy dane ze 100 stron, co daje nam zbiór 10 tys. gier. Domyślnie gry są posortowane według oceny, zatem sortujemy je po liczbie ocen malejąco, aby uzyskać jak najbardziej wiarygodne wyniki.

Odwiedzamy kolejne strony rankingu, używając adresów URL zawierających numer strony. Linki są zapisywane w pliku `links.txt`, aby uniknąć ponownego pobierania w przypadku przerwania procesu.

Pobieranie danych o grach

Metoda `get_games` iteruje przez pobrane linki i dla każdej gry pobiera szczegółowe informacje. Dla każdej gry wykonano następujące kroki:

- Odwiedzano zakładkę z danymi statystycznymi (`/stats`) na stronie szczegółowej gry.
- Pobierano informacje o grze, w tym:
 - **Podstawowe dane:** nazwa gry, rok wydania, liczba graczy, czas rozgrywki, wymagany wiek, złożoność gry i alternatywne nazwy gry (jeśli istnieją).
 - **Opinie:** liczba ocen, średnia ocena, odchylenie standardowe oraz dokładny rozkład ocen w skali od 1 do 10.
 - **Statystyki:** liczba komentarzy, fanów, wyświetleń strony, liczba rozgrywek (ogółem i w bieżącym miesiącu).
 - **Informacje o posiadaniu:** liczba osób posiadających grę, liczba chętnych na zakup lub wymianę.
 - **Osoby zaangażowane:** projektanci, artyści i wydawcy.
- Wszystkie te informacje były wyodrębniane przy użyciu odpowiednich selektorów CSS i metod Selenium (`find_element` lub `find_elements`).
- Dane są zapisywane w formie słownika i dodawane do listy gier.
- W przypadku brakujących danych, skrypt zapisuje wartość `None`. Dodatkowa metoda `fill_missing` umożliwia ponowne odwiedzenie stron gier z brakującymi danymi w celu ich uzupełnienia (jeśli brak danych wynikał z błędu ładowania strony).

Zabezpieczenia i optymalizacja

- W przypadku problemów z ładowaniem strony (np. pusta strona lub opóźnienia serwera), skrypt wykonuje wielokrotne próby ponownego załadowania strony.
- Dla zabezpieczenia danych postęp scrapowania jest zapisywany regularnie, co umożliwia wznowienie pracy od ostatniego zarejestrowanego punktu kontrolnego.
- W celu zmniejszenia ryzyka blokady przez serwer BoardGameGeek, wprowadzono losowe opóźnienia między zapytaniami (od 2 do 10 sekund).

Zapisywanie danych

Dane są zapisywane w pliku `games.csv` co określoną liczbę gier (domyślnie co 100 gier), aby zapewnić bezpieczeństwo danych w przypadku przerwania procesu. Format CSV pozwala na łatwe wczytanie danych do analizy przy użyciu biblioteki `pandas`.

Zakończenie

Po zakończeniu procesu scrapowania, przeglądarka Chrome jest zamykana, a dane są gotowe do analizy.

Analiza danych

Strona [boardgamesgeek](#) ma 10-stopniową skalę oceniania gier planszowych. Gry oceniane są od 1 do 10, gdzie 1 oznacza najgorszą ocenę, a 10 najlepszą. Zbadamy cechy gier, które mają wpływ na ich lepszą ocenę oraz cechy, które wpływają na największą spójność ocen (czyli czy gra jest oceniana podobnie przez różnych użytkowników). Spróbujemy także stworzyć modele predykcyjne, które będą przewidywać jak może być oceniana gra na podstawie jej cech.

Struktura danych

Pierwszym krokiem analizy jest wczytanie danych z pliku `games.csv`, uzyskanego w wyniku scrapowania.

```
df = read_games()
df.shape
```

```
(10000, 36)
```

Mamy przygotowane dane 10 tys. wierszy i 36 kolumn, z których 6 to zmienne kategoryczne, a 30 to kolumny z danymi liczbowymi. Zebrane dane zawierają następujące kolumny:

1. Podstawowe informacje:

- Nazwa (name)
- Rok wydania (year)
- Opis (description)
- Wymagana liczba graczy (min_players, max_players)
- Czas gry (min_playtime, max_playtime)
- Wymagany wiek (min_age)
- Złożoność gry (complexity)
- Alternatywne nazwy (alternate_names)
- Designerzy (designers)
- Artyści (artists)
- Wydawcy (publishers)

2. Opinie:

- Liczba ocen (ratings)
- Średnia ocena (avg_rating)
- Odchylenie standardowe (std_rating)
- Rozkład ocen (ratings_1, ratings_2, ..., ratings_10)

3. Statystyki gry:

- Liczba komentarzy (comments)
- Liczba fanów (fans)
- Wyświetlenia strony (page_views)

4. Statystyki rozgrywek:

- Liczba rozgrywek (plays)
- Liczba rozgrywek w tym miesiącu (plays_month)

5. Dane na temat posiadania gry:

- Liczba osób posiadających grę (owners)
- Liczba osób, które posiadały grę (prev_owned)
- Liczba osób, które chcą sprzedać grę (for_trade)
- Liczba osób, które chcą kupić grę (want_in_trade)
- Liczba osób, które mają grę na liście życzeń (wishlist)

Czyszczenie i przetwarzanie danych

Usuujemy kolumnę `description` ponieważ nie wykorzystujemy jej w analizie.


```
df = df.drop(columns=['description'])
```

Usuwamy wiersze z brakującymi danymi.

```
df = df.dropna()
```

Konwertujemy zmienne na odpowiednie typy danych.

```
cols_float = ['complexity', 'avg_rating', 'std_rating']
cols_int = ['year', 'min_players', 'max_players', 'min_play_time', 'max_play_time',
            'min_age', 'ratings', 'ratings_1', 'ratings_2', 'ratings_3', 'ratings_4',
            'ratings_5', 'ratings_6', 'ratings_7', 'ratings_8', 'ratings_9',
            'ratings_10', 'comments', 'fans', 'page_views', 'plays', 'plays_month',
            'owners', 'prev_owned', 'for_trade', 'want_in_trade', 'wishlist']
cols_list = ['alternate_names', 'designers', 'artists', 'publishers']
df[cols_float] = df[cols_float].astype(np.float32)
df[cols_int] = df[cols_int].astype(np.int32)
df[cols_list] = df[cols_list].map(lambda x: [i.strip(',') for i in x])
```

Analiza danych

```
print_table(df.describe().T, headers='keys')
```

	count	mean	std	min	25%	50%	75%	max
year	9721	2006.23	120.418	-3500	2008	2015	2019	2025
min_players	9721	1.88633	0.723886	1	1	2	2	9
max_players	9721	5.69612	8.80932	1	4	4	6	100
min_play_time	9721	55.0787	89.0258	1	30	45	60	5400
max_play_time	9721	84.0628	239.606	1	30	60	90	12000
min_age	9721	10.8906	2.58212	2	8	12	13	21
complexity	9721	2.24183	0.799187	1	1.63	2.17	2.77	4.86
ratings	9721	2701.97	6748.38	324	505	896	2075	131155
avg_rating	9721	6.96794	0.797025	2.747	6.467	6.985	7.522	9.387
std_rating	9721	1.39917	0.204331	0.95	1.26	1.37	1.51	4.03
ratings_1	9721	10.5343	43.2513	1	2	3	7	2233

	count	mean	std	min	25%	50%	75%	max
ratings_2	9721	17.8127	68.173	1	3	7	15	4185
ratings_3	9721	37.2363	110.332	1	8	17	34	5894
ratings_4	9721	81.7612	199.653	1	20	41	79	8042
ratings_5	9721	176.245	348.026	1	51	94	174	8601
ratings_6	9721	477.891	1010.47	0	119	201	430	26381
ratings_7	9721	660.714	1637.91	0	114	221	544	37689
ratings_8	9721	780.537	2323.82	0	67	171	538	46243
ratings_9	9721	294.303	1090.56	0	17	52	173	28049
ratings_10	9721	168.302	778.607	0	4	22	86	24892
comments	9721	589.319	1130.14	29	159	264	536	22457
fans	9721	193.464	506.124	0	27	61	158	10951
page_views	9721	268752	504472	8836	71228	127706	258999	1.32543e+07
plays	9721	10048.1	36621.6	0	938	2131	5880	837775
plays_month	9721	46.4011	209.377	0	1	4	20	5933
owners	9721	5221.8	10370.4	27	1293	2380	5016	215312
prev_owned	9721	585.955	937.895	3	161	296	616	15111
for_trade	9721	109.746	158.979	0	34	63	123	3202
want_in_trade	9721	105.761	164.497	0	23	53	117	1983
wishlist	9721	703.45	1456.57	5	124	267	640	24480

- Cel analizy (np. zidentyfikowanie najpopularniejszych gier, analiza ocen w zależności od liczby graczy).
- Metody analizy (np. statystyki opisowe, wizualizacje).
- Szczegółowe wyniki:
 - Wykresy, tabele, wnioski.
 - Przykłady interesujących zależności.

Wyniki i wnioski

- Podsumowanie najważniejszych wyników analizy.
- Interpretacja wyników w kontekście tematyki gier planszowych.

Modelowanie predykcyjne

W przypadku predykcji oceny gry, nie będziemy korzystać z żadnej ze zmiennych związanych z ocenami, aby uniknąć tzw. „data leakage” (wycieku danych).

Przygotowanie danych

Podsumowanie

Wnioski końcowe

- Refleksje nad procesem realizacji projektu.
- Możliwości ulepszenia scrapera i analizy w przyszłości.

Zastosowania Scraperów:

1. Analiza gier planszowych

Dane zebrane przez scraper mogą być wykorzystane do analiz popularności, ocen i statystyk związanych z grami planszowymi.

2. Tworzenie rekomendacji

Informacje o ocenach i opiniach użytkowników mogą służyć do budowania systemów rekomendacyjnych.

3. Porównania i statystyki rynkowe

Zebrane dane umożliwiają identyfikację trendów w branży gier planszowych, popularnych projektantów czy wydawców.

4. Tworzenie baz danych

Dzięki zapisowi danych w pliku CSV, możliwe jest łatwe tworzenie własnych baz danych do dalszych analiz.